

Monitoring an Apache Spark Job Using Prometheus and Grafana

Project Documentation

Samsung Innovation Campus

1. Introduction

This document describes a project in which an Apache Spark application is developed, executed with spark-submit, and monitored with Prometheus and Grafana. The application processes customer and order data, while the monitoring stack collects and visualizes Spark execution and performance metrics.

1.1 Objective

The objective of the project is to build a simple Apache Spark application, run it using spark-submit, and monitor its execution and performance using Prometheus and Grafana.

1.2 Scope

The project covers the following activities:

- Developing a PySpark application named spark_job.py that reads and processes two CSV datasets.
- Executing the application with spark-submit.
- Configuring Spark to expose metrics that Prometheus can collect, and verifying that Prometheus receives them.
- Adding Prometheus as a data source in Grafana.
- Creating a Grafana dashboard named Spark Job Monitoring Dashboard that contains at least six panels.

2. Technologies and Environment

2.1 Technologies

- Apache Spark 4.2.0 (Scala 2.13.18), installed through the PySpark 4.2.0 package.
- Python 3.13.12.
- OpenJDK 17.0.20.1 (Temurin).
- Docker 29.2.1, used to run Prometheus and Grafana through Docker Compose.
- Prometheus (Docker image prom/prometheus).
- Grafana (Docker image grafana/grafana).

2.2 Operating Environment

The project was developed and executed on a Windows machine. The Spark application ran in local mode, and Visual Studio Code was used to create the project files and run the commands. Prometheus and Grafana ran as Docker containers on the same machine.

3. Project Structure

All project files are located in a single directory named `spark-monitoring`, which contains the following files:

- `customers.csv`: the customer dataset.
- `orders.csv`: the order dataset.
- `spark_job.py`: the PySpark application.
- `metrics.properties`: the Spark metrics configuration.
- `prometheus.yml`: the Prometheus scrape configuration.
- `docker-compose.yml`: the Docker Compose definition for Prometheus and Grafana.

4. System Architecture

The system consists of three cooperating components arranged as a monitoring pipeline: Spark, Prometheus, and Grafana.

- Spark executes the data-processing job and exposes its metrics over HTTP through the Spark UI endpoint on port 4040.
- Prometheus periodically scrapes the Spark metrics endpoints and stores the collected metrics.
- Grafana queries Prometheus and displays the metrics on a dashboard.

Prometheus and Grafana run in Docker containers and reach the Spark application, which runs on the host machine, through the `host.docker.internal` address.

5. Implementation

5.1 Datasets

The application reads two CSV files, each with a header row. Their columns, as observed in the provided files, are listed below.

`customers.csv` contains the following columns:

- `customer_id`

- name
- country
- age

orders.csv contains the following columns:

- order_id
- customer_id
- product
- category
- quantity
- price
- status

5.2 Spark Application

The application `spark_job.py` creates a Spark session named `SparkJobMonitoring` and reads both CSV files with schema inference enabled. It then performs the following processing steps:

1. Retain only customers whose age is greater than or equal to 18.
2. Retain only orders whose status is "Completed". Leading and trailing whitespace in the status value is trimmed before comparison.
3. From the cleaned orders, retain only orders whose quantity is greater than or equal to 2.
4. Join the filtered orders with the filtered customers on `customer_id`.
5. Compute the revenue of each order as quantity multiplied by price.
6. Group the results by country and sum the revenue to obtain `total_revenue`.
7. Sort the result from the highest to the lowest `total_revenue` and display it.

After displaying the result, the application pauses for 300 seconds before stopping the Spark session. This pause keeps the Spark UI and its metrics endpoints available so that Prometheus can scrape them and Grafana can display them while the application is still running.

5.3 Running the Application

The application is executed with `spark-submit`. The following options enable the Prometheus metrics endpoint, load the metrics configuration file, and set the metrics namespace:

```
spark-submit --conf spark.ui.prometheus.enabled=true --conf
spark.metrics.conf=metrics.properties --conf spark.metrics.namespace=spark
spark_job.py
```

On Windows, the Scripts directory of the Python installation was not on the system PATH, so the spark-submit.cmd file located in the bin directory of the installed pyspark package was added to the PATH of the terminal session and invoked as spark-submit.cmd.

5.4 Spark Metrics Configuration

The file metrics.properties enables the PrometheusServlet sink for all Spark metric instances and assigns a separate HTTP path to each of them:

- Driver metrics: /metrics/driver/prometheus
- Executor metrics: /metrics/executors/prometheus
- Master metrics: /metrics/master/prometheus
- Applications metrics: /metrics/applications/prometheus

These endpoints are served by the Spark UI on port 4040 while the application is running.

5.5 Prometheus Configuration

The file prometheus.yml sets a global scrape interval of five seconds and defines two scrape jobs, both targeting host.docker.internal:4040:

- spark-driver: scrapes the path /metrics/driver/prometheus/.
- spark-executors: scrapes the path /metrics/executors/prometheus/.

5.6 Docker Compose Configuration

The file docker-compose.yml defines two services:

- prometheus: uses the prom/prometheus image, publishes port 9090, and mounts prometheus.yml as its configuration file.
- grafana: uses the grafana/grafana image and publishes port 3000.

Both services map host.docker.internal to the host gateway so that the containers can reach the Spark application on the host machine. The services are started with the command docker compose up -d.

5.7 Grafana Configuration

Prometheus was added as a data source in Grafana using the server URL http://prometheus:9090, and the connection was confirmed with the Save and test function, which reported that the Prometheus API was queried successfully.

6. Grafana Dashboard

A single dashboard named Spark Job Monitoring Dashboard was created. It contains six panels, described below with the Prometheus query used for each.

6.1 Running Jobs

Displays the number of currently running Spark jobs.

- Query: `metrics_spark_driver_DAGScheduler_job_activeJobs_Number`

6.2 Completed Jobs

Displays the number of completed Spark jobs. Because the driver does not expose a dedicated completed-jobs metric, the value is calculated as the total number of jobs minus the number of active jobs.

- Query: `metrics_spark_driver_DAGScheduler_job_allJobs_Number - metrics_spark_driver_DAGScheduler_job_activeJobs_Number`

6.3 Failed Jobs

Displays the number of failures. Spark does not expose a ready-made failed-jobs metric in the available endpoints, so the panel uses the number of failed stages as the closest available indicator.

- Query: `metrics_spark_driver_DAGScheduler_stage_failedStages_Number`

6.4 CPU Usage

Displays the CPU utilization of the Spark application, estimated as the rate of the total task execution time reported by the executor. The panel unit is a percentage on a scale of 0.0 to 1.0.

- Query: `rate(metrics_executor_totalDuration_seconds_total[1m])`

6.5 Executors

Displays the number of Spark executors.

- Query: `count(metrics_executor_totalCores)`

6.6 JVM Heap Memory

Displays the JVM heap memory used by the executor, formatted in bytes (IEC units). This panel was added as the sixth panel to satisfy the minimum of six panels.

- Query: `metrics_executor_JVMHeapMemory_bytes`

7. Results

7.1 Data Processing Output

The application produced the following total revenue per country, sorted from the highest to the lowest value:

- Egypt: 122485
- France: 52786
- USA: 38068
- Canada: 30753
- Germany: 20084
- UAE: 18115
- India: 11559
- UK: 11164
- Saudi Arabia: 9545

7.2 Prometheus Verification

The Prometheus Target health page reported both scrape targets, spark-driver and spark-executors, in the UP state, confirming that Prometheus successfully receives Spark metrics.

7.3 Dashboard Values

While the application was running, the dashboard displayed the following values:

- Running Jobs: 0
- Completed Jobs: 7
- Failed Jobs: 0
- CPU Usage: 43.5%
- Executors: 1
- JVM Heap Memory: 164 MiB

Because the application runs in local mode, the single executor reported is the driver process itself.

8. Challenges and Solutions

- The spark-submit command was not recognized after installing PySpark with pip. It was resolved by invoking spark-submit.cmd from the bin directory of the installed pyspark package and adding that directory to the PATH of the terminal session.
- The spark-driver target initially returned HTTP 404 Not Found because the configured path /metrics/prometheus/ did not match the driver path defined in metrics.properties. Changing the scrape path in prometheus.yml to /metrics/driver/prometheus/ brought the target to the UP state.
- No ready-made completed-jobs or failed-jobs metrics were available. The Completed Jobs panel was derived from the total and active job counts, and the Failed Jobs panel uses the failed stages metric.
- Running the application more than once produced several application_id values, so the CPU Usage and JVM Heap Memory panels displayed more than one series. Aggregating the queries with sum() was identified as a way to obtain a single value.

9. Limitations

- The application runs in local mode, so no separate executors exist and the executor count is always one.
- Spark metrics are available only while the application and its Spark UI are running; the application therefore pauses for 300 seconds before stopping.
- The Failed Jobs panel is based on failed stages rather than failed jobs.
- The CPU Usage panel is an estimate derived from task execution time, not a direct measurement of CPU utilization.

10. Conclusion

The project met its objective. A PySpark application was developed and executed with spark-submit to compute total revenue per country from filtered customer and order data. Spark was configured to expose its metrics, Prometheus was verified to receive them, and a Grafana dashboard named Spark Job Monitoring Dashboard with six panels was created to display job counts, failures, CPU usage, executor count, and JVM heap memory.